0036洛谷P5732 杨辉三角

```
#include<iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int a[21][21] = { 0 };
```

我们之前用的a[]是一维数组，这题中的a[][]是二维数组

二维数组就是带行和列的表格，用 a[行][列] 访问，行和列都从0开始数。

处理矩阵、棋盘地图、DP表格这类“平面数据”时用它；三维以上极少用，一般把问题拆成二维结构体。

多维数组本质就是“数组里套数组”，几维就需要几层循环来遍历。

```
    for (int i = 0; i < n;i++) {
        for (int j = 0; j <= i;j++) {
```

像这种有二维平面的题，适合考虑二维数组，可以用a[i][j]来表示当前数据,i和j就是该数据在数组中对应的位置

```
            if (j == 0 || j == i) {
                a[i][j] = 1;
```

当某一数据位于最左端和最右端时，赋值为1，防止越界

```
            }
            else {
                a[i][j] = a[i - 1][j - 1] + a[i - 1][j];
```

在图中样例中，仔细观察可以发现其余数据的规律：其等于左端和上端的相邻数据之和

我们用a[i-1][j-1]表示其上端相邻数据，a[i-1][j]表示其左端的相邻数据，使其相加得到当前数据

什么？你问我左端和上端的数据是怎么的来的？

因为我们赋值好了最左端和最上端的数据为1，通过不断相加得来的

```
            }
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

完整代码

```
#include<iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
```

```
    int a[21][21] = { 0 };

    for (int i = 0; i < n;i++) {
        for (int j = 0; j <= i;j++) {

            if (j == 0 || j == i) {
                a[i][j] = 1;

            }
            else {
                a[i][j] = a[i - 1][j - 1] + a[i - 1][j];

            }
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```